C++ Questions for an HFT Systems Interview

Targeted at a hands-on low-latency CTO — matching engine, feed handler, execution

- Why these questions
- Part 1 — Matching Engine
- Part 2 — Feed Handler
- Part 3 — Execution and OMS
- Part 4 — C++ Specifics He's Likely to Probe
- Part 5 — Where You'll Say "I Haven't Done That"
- Questions to ask him

Why these questions

The interviewer builds HFT, OMS and algo platforms — ExaNIC, Solarflare, RHEL, multiple exchange connections. He will not ask textbook definitions. He will ask **what you did, why, and what it cost**, and he will follow up until he finds the edge of what you actually know.

Your overlap is large: matching engine, feeder, execution, paper trading, strategy. Your boundary is the NIC and bypass layer. These questions are aimed at the overlap.

Part 1 — Matching Engine

He will probe here hardest, because it's the deepest systems work on your CV.

Q: How did you structure the order book?

Start with the constraint, not the container: the book needs O(1) access to the best bid and ask, fast insert and cancel at arbitrary levels, and iteration in price order.

The common design is a **two-level structure**:

1. **Price levels** — an array indexed by price (offset from a base tick), or a sorted map. An array is O(1) and cache-friendly, but only works when the price range is bounded and known. A `std::map` handles arbitrary ranges but pointer-chases.
2. **Orders within a level** — an intrusive doubly-linked list, in arrival order, for price-time priority.

Plus a **hash map from order id to a node pointer**, so a cancel is O(1) — you jump straight to the node and unlink it, without searching the level.

Why intrusive: the list node lives *inside* the order struct, so there's no separate allocation per list insertion and no extra indirection. Order objects come from a preallocated pool.

If you didn't build it exactly this way, say what you did and why. The reasoning matters more than matching a canonical design.

Q: Array of price levels or a map — how do you choose?

Array wins when the price range is bounded. Indian equities have circuit limits, so the day's range is known at open — you can allocate a slab covering it and index directly. O(1), contiguous, prefetch-friendly.

Map wins when the range is unbounded or extremely sparse, at the cost of a pointer chase per lookup.

The hybrid many desks use: **an array for a window around the touch, spilling to a map for far levels.** Almost all activity is near the touch, so you get array speed where it matters.

The follow-up to expect: *"How big is the array?"* If it's 20,000 levels × 8 bytes that's 160 KB — beyond L2. So keep the

level struct small and store hot fields (total quantity, order count) separately from cold ones.

Q: How do you cancel an order in O(1)?

Hash map from order id to the node. Because the list is intrusive and doubly-linked, unlinking is just fixing two pointers — no search.

The subtlety is **what happens to the empty level**. If you delete the level object every time it empties and recreate it on the next order, you're allocating on a hot path. Better to leave the level allocated with a zero count and reclaim lazily, or never.

Q: How would you handle order modify?

Depends on the exchange's rules, and this is a good thing to say — it's a business rule, not a data structure question.

- **Price change, or quantity increase** → loses time priority. Cancel and re-insert at the back of the new level.
- **Quantity decrease** → typically keeps priority. Just decrement in place.

Getting this wrong makes your simulator diverge from live in a way that's very hard to notice.

Q: What was the throughput, and where did the time go?

Have real numbers if you can share them. If you can't, give the shape:

The dominant costs in a matching engine are usually (a) the hash lookup on cancel, (b) cache misses walking to a price level, and (c) any allocation you didn't eliminate. The matching itself — comparing prices and decrementing quantities — is trivial arithmetic.

The strong version of this answer names how you found out, not just the number: *"I profiled with perf stat first; the branch-miss rate told me it was X rather than Y."*

Q: How do you avoid allocation in the matching path?

Preallocate an object pool of order structs at startup, sized to the maximum outstanding orders. Allocation becomes taking the head of a free list; freeing is pushing it back. Both O(1), no lock, no page fault.

The failure mode to mention: **pool exhaustion**. You need a policy — reject the order, or grow at a safe moment. Growing silently mid-session reintroduces exactly the allocation you removed.

Part 2 — Feed Handler

Q: Walk me through your feed handler.

Layers: receive → decode → book update → publish.

Key decisions at each layer:

- **Receive** — one thread per multicast group, or one thread handling several? Busy-poll or epoll? (If you were above the socket layer, say so — that's the boundary.)
- **Decode** — the wire format is usually fixed-width binary. Cast a packed struct over the buffer rather than field-by-field parsing, watching alignment and endianness.
- **Book update** — apply the delta.
- **Publish** — how do strategies see it? A callback (simple, but a slow consumer stalls the feed), or a shared-memory ring per consumer (decoupled, and one slow reader can't hurt others).

Q: How do you handle sequence gaps?

Every exchange feed carries a sequence number. On a gap you have three options, in order of preference:

1. **Arbitration** — most exchanges publish two identical multicast feeds (A and B) on separate paths. Take whichever packet arrives first, and the other covers a drop. This handles the overwhelming majority of gaps at zero latency cost.
2. **Retransmission request** — ask for the missing packets. Slower.
3. **Snapshot recovery** — rebuild the book from a periodic snapshot feed, then replay incrementals from the snapshot's sequence number.

The important operational point: while you're in recovery, your book is **wrong**, and the strategy must know that. Publish a "book is stale" state rather than continuing to quote off bad data. That's the kind of answer that shows you've thought about failure, not just the happy path.

Q: Snapshot versus incremental — how do they fit together?

Incrementals are deltas and are the normal path. Snapshots are periodic full images, used to build the book at start-of-day or after an unrecoverable gap.

The join is the tricky part: you buffer incrementals while the snapshot arrives, then discard any buffered message with a sequence number at or below the snapshot's, then apply the rest. Getting that boundary wrong gives you a book that's subtly wrong all day.

Q: How do you get decoded data to the strategy threads?

If it's one consumer: **SPSC ring buffer** in shared memory. Lock-free, and the writer never blocks.

If it's several: **one ring per consumer**, written by the feed thread. Costs a copy per consumer, but keeps them fully independent — one slow strategy can't stall the feed or the other strategies.

The alternative — a single shared buffer with each reader tracking its own position — avoids the copy but means a slow reader can be lapped. Then you need to detect that and tell it.

This is where a sequence-number-per-slot design earns its keep — the reader checks the sequence before and after copying, and knows if it was overwritten mid-read.

Q: Why not just use a mutex around the book?

Because a reader that gets descheduled by the kernel while holding the lock stalls the **feed thread** for a full scheduling quantum. Then the NIC ring backs up and you drop packets.

The market data thread must never wait on a calculation thread. That constraint alone rules out mutexes on this path, before you count a single nanosecond.

And I have measured this: a mutex also gives exclusion without **order**. Same fills across four threads under one mutex gave 30 distinct results from 30 identical runs — the share counts agreed every time because addition commutes, while the P&L didn't. A smoke test passes while the money numbers are wrong.

Part 3 — Execution and OMS

Q: How do you track order state?

A state machine per order: `New → PendingNew → Acknowledged → PartiallyFilled → Filled` / `Cancelled` / `Rejected`, with `PendingCancel` and `PendingReplace` as intermediate states.

The hard part is the race: you send a cancel, and a fill for that order arrives before the cancel acknowledgement. You cannot treat the order as cancelled until the exchange says so — the exchange's view is authoritative, always.

Say that explicitly. It's the difference between someone who has run an OMS and someone who has drawn one.

Q: How do you reconcile your position with the exchange?

Never assume your local state is right. Reconcile against exchange drop-copy or end-of-day files. Any discrepancy is an incident, not a rounding issue.

Common causes: a fill you missed during a disconnect, a partial fill counted twice on reconnect, or an order you thought was cancelled that traded.

Q: How do you compute average entry price and P&L?

This one you can answer from the work you just did.

Track a signed quantity and a **cost basis**, and derive the average price. Not the other way round.

Storing the average and reconstructing the cost is backwards, and it's where floating-point drift comes from — I measured 200,000 accumulating blends drifting from 0.3333333333333333 to 0.33333333333395226. Storing an exact integer cost basis in price ticks makes it exact, and makes a conservation property provable: realised plus remaining cost equals cash paid, at every point.

Realised P&L is booked only when a fill reduces the position — opens and adds book nothing. The remaining position is marked against an **externally supplied mark**, not the last fill price and not its own entry price.

And the convention matters: weighted average, FIFO and LIFO give different realised/unrealised splits but the **same total**. India mandates FIFO for tax, so a weighted-average risk book will correctly disagree with the tax book. Reconcile on total.

Q: How do you make a risk check fast enough to sit in the order path?

Precompute everything you can. Position limits, notional limits and fat-finger bounds are just integer comparisons against values updated on fill, not recomputed per order.

Keep the checks branch-predictable — the reject path is `[[unlikely]]` and almost never taken.

Never allocate, never log-with-formatting, never take a lock in the check. If a check needs to record a rejection, push a binary record into a queue and format it elsewhere.

Part 4 — C++ Specifics He's Likely to Probe

Q: How do you decode a binary wire protocol in C++ without copying?

Define a packed struct matching the layout and point it at the buffer:

```
#pragma pack(push, 1)
struct AddOrderMsg {
    uint8_t  type;
    uint64_t order_id;
    uint32_t price;
    uint32_t qty;
};
#pragma pack(pop)
```

Four things to raise, unprompted — this is where he's testing:

1. **Alignment.** Casting a `char*` to a struct pointer is technically UB (strict aliasing and alignment). `memcpy` into a local is the defined way and compiles to the same thing at `-O2`. In C++20, `std::bit_cast`.
2. **Endianness.** Network order is big-endian; x86 is little. `be64toh` and friends.
3. **Packing.** `#pragma pack` removes padding but creates unaligned members — fine on x86, a fault on some ARM.
4. **Versioning.** Exchanges change formats. You need the message version in the header and a plan.

Q: How do you time something at nanosecond resolution?

`rdtsc` / `rdtscp` — a few cycles, versus ~20–25 ns for `clock_gettime`.

Caveats to volunteer: `rdtsc` can be reordered by the CPU (`rdtscp` serialises), you must calibrate the TSC frequency to convert to nanoseconds, and you need an invariant TSC — otherwise frequency scaling breaks it.

Then say the important part: report percentiles, never a mean. p50, p99, p99.9, max. A mean hides exactly the tail that matters, and in trading the tail *is* the risk.

Q: You have a hot loop that's slower than you expect. Walk me through what you do.

`perf stat` first. Three numbers tell you the shape immediately:

- **Low IPC** → stalling, usually memory
- **High LLC-load-misses** → memory bound, look at layout
- **High branch-misses** → mispredict bound, look at the control flow

Then `perf record` / `report` for attribution by symbol. `perf c2c` if it's threaded, for false sharing.

And one habit worth mentioning: check that the component timings add up to the end-to-end number. When they don't, the benchmark is lying. That mismatch is how I found a memory-layout bug in my own code recently — components summed to 4 µs but end-to-end was 18 µs, and chasing the gap found I'd laid out my carried state stock-major when the access pattern was cross-sectional.

Q: How would you lay out a struct for cache efficiency?

Order members largest to smallest to minimise padding. Keep hot fields together; move cold fields to a separate array. Check the total size against the cache line — a struct straddling a boundary costs two fetches.

`alignas(64)` on anything written by different threads, to avoid false sharing.

Measured example: changing a price matrix from `[stock][day]` to `[day][stock]` gave **3×** — 1,017 ns to 339 ns — because the access was cross-sectional, so the first layout fetched 500 cache lines and used 16 bytes of each 64.

Q: When would you use a template rather than a virtual function here?

When the type is known at compile time and the call is on the hot path. A virtual call costs two dependent loads and an indirect branch, but the real cost is that it **blocks inlining** — you lose all downstream optimisation across it.

Always give the cost too: code bloat (one instantiation per type, so I-cache pressure), slow compiles, everything in headers, and — the practical one — **you can't read the parameter from a config file.**

I made that call deliberately recently: a window size stayed a runtime config field rather than a template parameter, precisely so it could be tuned per deployment.

Q: Show me a lock-free SPSC queue.

```cpp
template <typename T, size_t N>           // N a power of two
class SpscQueue {
    static_assert((N & (N - 1)) == 0);
public:
    bool push(const T& item) {
        size_t w = write_.load(std::memory_order_relaxed);   // only we write it
        size_t next = (w + 1) & (N - 1);
        if (next == read_.load(std::memory_order_acquire)) return false;   // full
        buffer_[w] = item;
        write_.store(next, std::memory_order_release);       // publish
        return true;
    }
    bool pop(T& out) {
        size_t r = read_.load(std::memory_order_relaxed);
        if (r == write_.load(std::memory_order_acquire)) return false;     // empty
        out = buffer_[r];
        read_.store((r + 1) & (N - 1), std::memory_order_release);
        return true;
    }
private:
    alignas(64) std::atomic<size_t> write_{0};
    alignas(64) std::atomic<size_t> read_{0};
    T buffer_[N];
};
```

Narrate as you write: each index has exactly one writer so its own load is `relaxed`; release publishes the payload; acquire makes the other side's writes visible; `alignas(64)` on both or the producer and consumer false-share one line; power-of-two turns modulo into a mask.

Q: What does acquire-release actually guarantee?

They pair **on the same variable**. A release store means everything written before it becomes visible to anyone who acquire-loads that variable. An acquire load means you can see everything the releasing thread did before its store.

On x86-64 acquire and release are plain `mov` instructions — free, because x86 won't reorder store-store or load-load. Only `seq_cst` stores cost anything (`mfence`, ~20–30 cycles). On ARM they're real instructions, so code that's free on x86 costs on Graviton.

Q: How do you know a race is actually fixed?

"It stopped reproducing" isn't evidence — a race made 100× rarer looks identical to one that's fixed.

In order of strength: Clang `-Wthread-safety` with `GUARDED_BY` annotations, which turns "accessed without its lock" into a **compile error** — a proof over all paths, not a sample. Then a model checker for lock-free code. Then ThreadSanitizer, which catches races on paths that execute — with the caveat that it **cannot model standalone `atomic_thread_fence`**, so a fence-based seqlock can't be validated that way. Stress testing last, as corroboration.

And run every check against the old code first. If TSan doesn't flag the original, your harness isn't reaching the racy path.

Part 5 — Where You'll Say "I Haven't Done That"

Prepare these, so they come out cleanly rather than as hesitation.

If asked about	Say
Solarflare / Onload / ExaNIC	*“That was a separate team. I worked above the socket layer.”*
Kernel tuning, isolcpus, IRQ affinity	*“I know what each does and why, but I haven’t been the one configuring it.”*
FPGA	*“No exposure.”*
Colocation setup	*“I understand the architecture, but I haven’t racked or configured it.”*

The pattern that works: name the boundary, then redirect to the adjacent thing you *do* know.

> *“I haven’t configured the NIC side — that was a separate team. What I worked on was the matching engine, the feed handler above the socket layer, execution and strategy. I know the tick-to-trade path conceptually and where the latency budget sits, but I’d be learning the bypass stack from you rather than bringing it.”*

Say it once, early, without apology. Then stop returning to it.

The failure mode is not admitting ignorance — it’s admitting it and then going silent. Where you can reason toward an answer, do:

> *“I haven’t used that. My guess would be X, because Y — but I’d want to verify rather than assume.”*

For a hands-on CTO, that reasoning is most of what he’s testing. He doesn’t need someone who has memorised his stack; he built it. He needs someone who can work out an unfamiliar problem in front of him and be straight about the edges.

Questions to ask him

- What does the tick-to-trade path look like today, and where’s the current bottleneck?
- Which exchanges are you live on, and which is next?
- Is the stack built in-house, or are you on a vendor OMS?
- ExaNIC or Solarflare — what drove that choice?
- How do you test strategy changes before they go live?
- What would you want me to have shipped in the first 90 days?

The last one is the best question you can ask a founding CTO. It tells you what the job actually is, and it’s hard to answer vaguely.